

Reporte Hormigas de Langton

Eduardo Hernández Castro

Complex Sytems – GENARO JUAREZ MARTINEZ – Grupo 7CM4

1 Introducción

En este documento se registran las observaciones y fundamentos técnicos de la simulación de hormigas basada en las reglas de Langton y el comportamiento de los distintos tipos de hormigas.

2 Fundamento del programa

La simulación está diseñada para evaluar cómo interactúan cuatro tipos de hormigas en una grilla con reglas de movimiento de Langton y ciclo de vida controlado. Cada sección del código aporta una parte clave del modelo.

2.1 Reglas principales

- Cada hormiga vive hasta 80 iteraciones. Esta regla se define en `config.py` mediante la constante `LIFESPAN` y se aplica en `simulation.py` durante la fase de envejecimiento y muerte.
- El estado de cada celda es negro/blanco, pero ese estado se maneja internamente y no se muestra en la visualización. El estado oculto se representa en `grid.py` con el arreglo `cell_state`.
- En celda negra la hormiga gira a la izquierda; en celda blanca gira a la derecha. Esta lógica está en `grid.py` dentro de la función `apply_langton_rule`.
- Después de girar, la hormiga invierte el color de la celda y avanza una celda. El volteo de la celda se hace con `flip_cell_state` en `grid.py` y el movimiento con `move_ant` o `attempt_movement_with_collision`.
- Si la celda destino está ocupada, la hormiga prueba hasta tres direcciones alternativas de forma aleatoria. Esta reintención se gestiona en `grid.py` dentro de `attempt_movement_with_collision`.
- Si todas las direcciones libres están ocupadas, la hormiga espera y no se mueve en esa iteración. El ant permanece en su posición cuando `attempt_movement_with_collision` no encuentra destino libre.

- Una hormiga nueva nace cuando una reproductora y una reina se enfrentan en direcciones opuestas y se cumple la probabilidad de reproducción. Esta interacción se evalúa en `simulation.py` durante la fase de reproducción, usando `REPRODUCTION_PROBABILITY` y `OPPOSITE_DIRECTIONS` desde `config.py`.
- Se detectan escenarios de aislamiento, sobrepoblación y estabilidad con umbrales de población y ocupación. Estas condiciones se definen en `config.py` y se usan en `simulation.py` dentro de `detect_scenariio`.

3 Estructura del código

La aplicación está organizada en módulos claros que separan la lógica de configuración, el modelo de datos, la simulación y la visualización.

3.1 Configuración

El archivo `config.py` contiene las constantes que determinan el comportamiento general:

- `GRID_SIZE`, `OCCUPANCY_RATIO` y `MAX_ANTS` para la dimensión y el número de hormigas iniciales.
- `ANT_DENSITIES` para la distribución inicial de los tipos de hormiga: 1% reinas, 55% trabajadoras, 9% reproductoras y 35% soldados.
- `LIFESPAN` define las 80 iteraciones de vida máxima de cada hormiga.
- `REPRODUCTION_PROBABILITY` y `OPPOSITE_DIRECTIONS` controlan cuándo se puede generar una nueva hormiga.
- `ISOLATION_THRESHOLD`, `OVERPOPULATION_GRID_THRESHOLD`, `OVERPOPULATION_DEATH_RATE`, `STABLE_POPULATION_VARIANCE` y `STABLE_GENERATIONS_REQUIRED` fijan los umbrales de escenario.

3.2 Modelo de hormigas

El módulo `ant.py` define la clase base `Ant` y las subclases `Queen`, `Worker`, `Reproducer` y `Soldier`. Cada hormiga mantiene su posición, dirección, edad y color de visualización. El método `age_one_iteration()` incrementa la edad y `is_alive()` determina si la hormiga debe morir cuando supera la longevidad.

3.3 Grilla y reglas de Langton

El módulo `grid.py` implementa la grilla de ocupación y el estado oculto de las celdas. Los componentes más relevantes son:

- `occupancy`: matriz booleana que indica qué celdas tienen hormigas.
- `cell_state`: matriz de valores 0/1 que representa el color oculto de cada celda.
- `apply_langton_rule`: aplica la regla de giro y calcula la nueva posición.

- `flip_cell_state`: cambia el color de la celda actual después del giro.
- `move_ant`: mueve la hormiga a una celda vacía.
- `attempt_movement_with_collision`: gestiona choques y reintentos en hasta tres direcciones adicionales.
- `get_occupancy_ratio`: calcula el porcentaje de celdas ocupadas por hormigas.

3.4 Simulación

El módulo `simulation.py` coordina la ejecución completa:

- `_initialize_ants`: crea la configuración inicial, asignando posiciones aleatorias y tipos de hormiga según `ANT_DENSITIES`.
- `iterate`: ejecuta el ciclo de movimiento, reproducción, envejecimiento y muerte por generación.
- `detect_scenario`: clasifica el estado actual como aislamiento, sobrepoblación o estabilidad.
- `run`: itera el número especificado de generaciones.
- Estadísticas por generación: total de hormigas, conteos por tipo, nacimientos, muertes, edad promedio y ocupación.

3.5 Visualización y reporte

El módulo `visualization.py` produce la salida gráfica y los resúmenes:

- Gráficos de evolución poblacional y ocupación.
- Guardado de imágenes iniciales y finales del grid.
- Exportación de un archivo de resumen `summary.txt` con las métricas clave de cada corrida.

3.6 Entrada principal

El archivo `main.py` expone el CLI para ejecutar la simulación con parámetros ajustables:

- número de iteraciones,
- tamaño de la grilla,
- ocupación inicial,
- modo toroidal,
- generación de reportes y CSV,
- registro de salida en archivo.

4 Escenarios

Según el código en `simulation.py` y los umbrales definidos en `config.py`, los escenarios se clasifican así:

1. Aislamiento: ocurre cuando la población actual baja por debajo del 10% de la población inicial. En nuestro caso, con 5000 hormigas iniciales, aislamiento se detecta si quedan menos de 500 hormigas.
2. Sobre población: ocurre cuando la ocupación de la grilla excede 80% y la tasa de muerte de la iteración es mayor al 15%. Esto implica una saturación del espacio combinada con un aumento rápido de defunciones.
3. Sistema estable: ocurre cuando el total de hormigas se mantiene con una varianza relativa de 5% o menos durante al menos 50 generaciones consecutivas.

5 Resultados

5.1 Caso 1 – Seed 29976446, 100x100, 200 generaciones

- Archivo de resumen: `S29976446_G100_I200_00.5_toroidal_20260605_074107_summary.txt`
- Generaciones: 200
- Hormigas finales: 1 (soldado)
- Ocupación final: 0.01%
- Hormigas en generación 80: 147
- Escenario detectado: aislamiento

5.2 Caso 2 – Seed 401529739, 100x100, 250 generaciones

- Archivo de resumen: `S401529739_G100_I250_00.5_toroidal_20260605_075905_summary.txt`
- Generaciones: 250
- Hormigas finales: 1 (trabajadora)
- Ocupación final: 0.01%
- Hormigas en generación 80: 183
- Escenario detectado: aislamiento

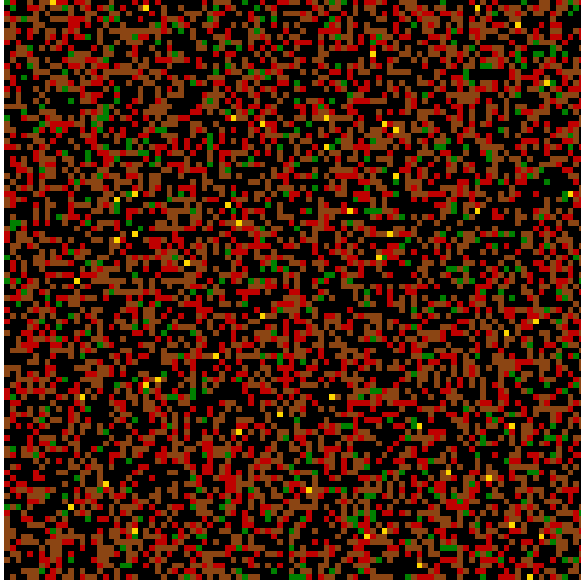


Figure 1: *
Caso 1: estado inicial

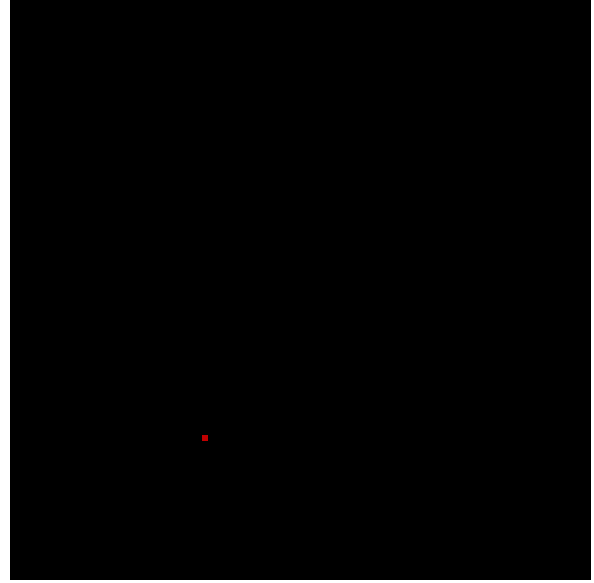


Figure 2: *
Caso 1: estado final

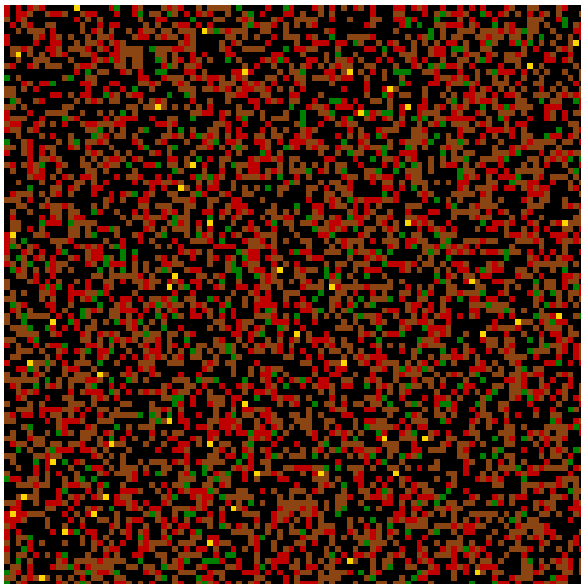


Figure 3: *
Caso 2: estado inicial

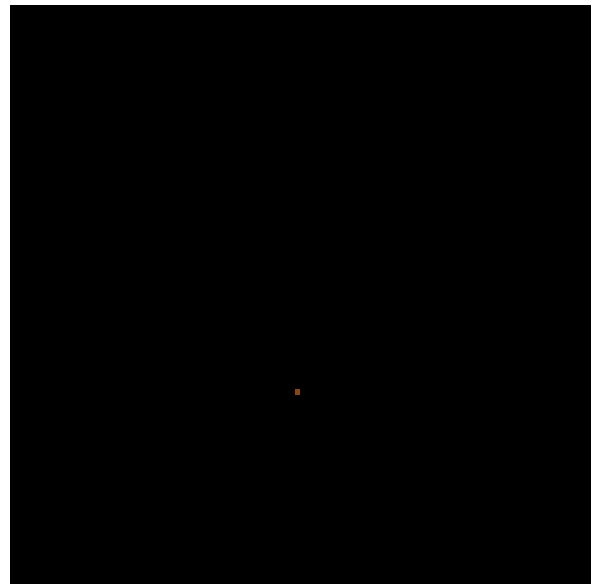


Figure 4: *
Caso 2: estado final

5.3 Notas de terminal y logs

El log `output/logs/find_seed_20260605_074103.txt` confirma la búsqueda de semilla y los resultados:

- Grid inicial 100x100 con 5000 hormigas y 50% de ocupación.
- Al finalizar 200 iteraciones quedó 1 hormiga y 0.01% de ocupación.
- En iteración 80 se tenía 147 hormigas.

5.4 Dependencia de la semilla y el tamaño del grid

La configuración inicial depende de la semilla aleatoria y del tamaño del grid. Al ser la grilla toroidal, dos tamaños de grid diferentes no pueden producir la misma configuración inicial válida, ya que la estructura de vecinos y la cantidad de celdas difieren.

6 Conclusiones

La evolución del sistema depende fuertemente de la semilla pseudoaleatoria usada para generar el estado inicial y de la incertidumbre inherente a respetar manualmente la ocupación y las proporciones de cada tipo de hormiga. En un grid de 100x100 con 50% de ocupación, colocar a mano miles de hormigas manteniendo las proporciones de reinas, trabajadoras, reproductoras y soldados no es práctico, y por eso el estado inicial se construye aleatoriamente a partir de una semilla. Esto hace muy difícil encontrar un estado inicial ideal en el que, por ejemplo, varias reproductoras queden correctamente ubicadas alrededor de las reinas y puedan reproducirse cada pocas iteraciones.

En los casos estudiados, incluso cuando una semilla produce más hormigas en la iteración 80, la ubicación de las hormigas que sobreviven más allá de las originales resulta decisiva. Las posiciones relativas de estas hormigas determinan si seguirán encontrándose con reproductoras y reinas, y por eso la diferencia entre los resultados de 200 iteraciones y 250 iteraciones no solo está en cuántas hormigas quedan, sino en cómo quedan distribuidas. Esto se aprecia en los archivos de salida `output/S401529739_G100_I250_00.5_toroidal_20260605_075905.png` y `output/S29976446_G100_I200_00.5_toroidal_20260605_074107.png`.

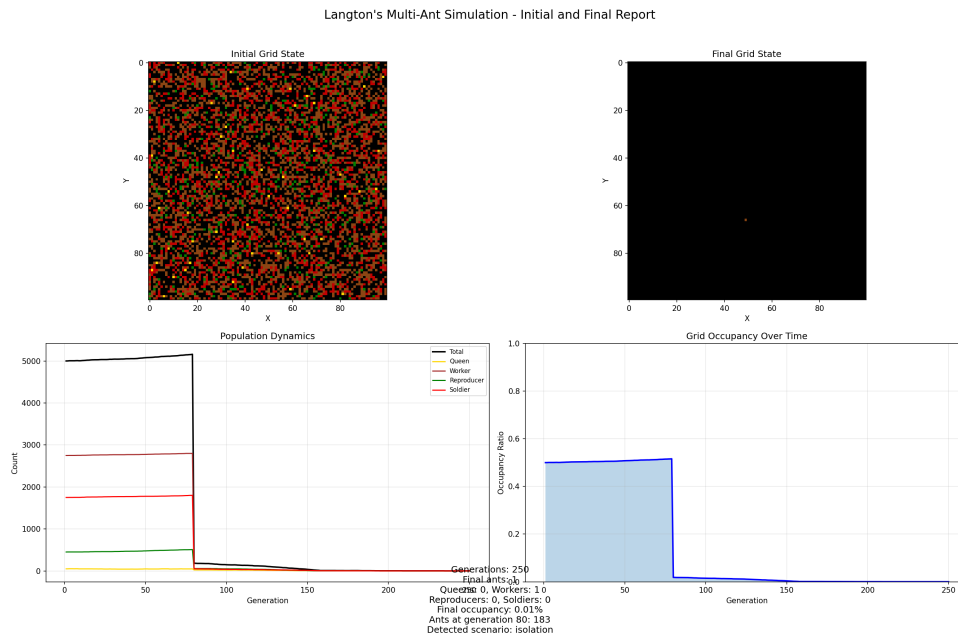


Figure 5: *
Caso 2: estado final en 250 generaciones

Otro efecto importante es que, en la mayoría de los casos, las hormigas nacidas después de la primera generación quedan demasiado separadas entre sí. Ese gran distanciamiento

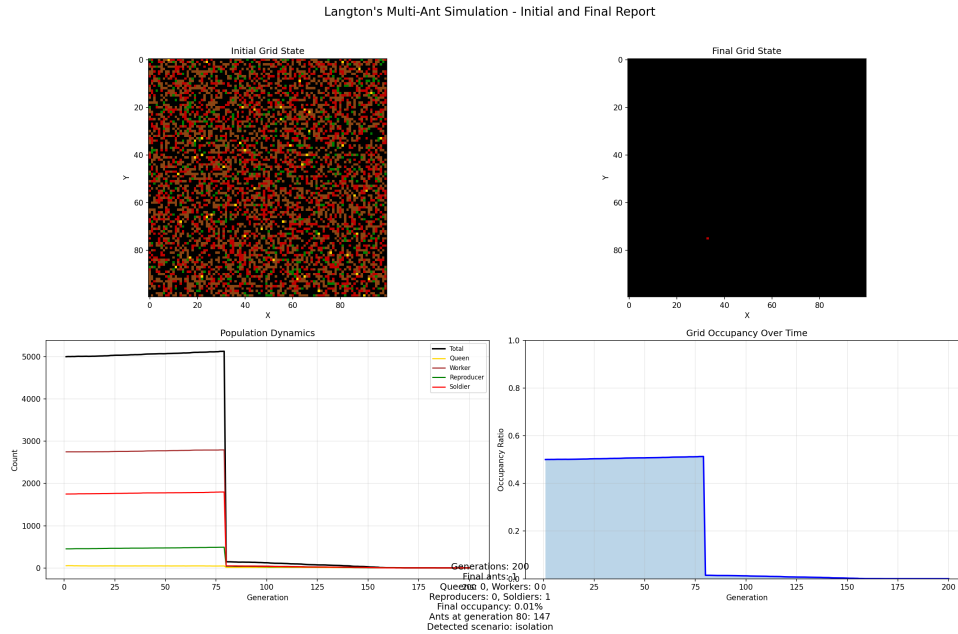


Figure 6: *
Caso 1: estado final en 200 generaciones

complica mucho que vuelvan a encontrarse más adelante, porque el movimiento de Langton tiende a dispersarlas y no a atraerlas. Por lo tanto, aunque haya un repunte en las generaciones intermedias, la distancia entre nacimientos reduce la probabilidad de que ocurran nuevos encuentros reproductivos y favorece la deriva hacia el aislamiento.

Estas observaciones refuerzan que el sistema es sensible a la configuración inicial y que los estados verdaderamente estables o de sobrepoblación son difíciles de lograr con generación aleatoria simple. Para avanzar, sería necesario explorar estrategias que controlen mejor la ubicación inicial de los individuos clave (reinas y reproductoras) o que introduzcan mecanismos adicionales para mantener las hormigas en áreas de encuentro más frecuentes.

7 Anexos

En esta sección se incluyen los códigos completos de los módulos principales de la simulación.

7.1 Archivo config.py

Listing 1: config.py

```
"""
Configuration constants for Langton's multi-ant simulation.
All values here can be adjusted for parameter tuning (requirement #8).
"""

import numpy as np

# =====
```

```

# GRID CONFIGURATION
# =====
GRID_SIZE = 100 # 100x100 grid by default
OCCUPANCY_RATIO = 0.5 # 50% of grid cells initially occupied (requirement #3)
MAX_ANTS = int(GRID_SIZE * GRID_SIZE * OCCUPANCY_RATIO)

# =====
# ANT TYPE DENSITIES (requirement #1 - initial distribution)
# =====
# Proportion of each ant type among all ants
ANT_DENSITIES = {
    'queen': 0.01, # 1% queens
    'worker': 0.55, # 55% workers
    'reproducer': 0.09, # 9% reproducers
    'soldier': 0.35 # 35% soldiers
}

# =====
# ANT PROPERTIES
# =====
LIFESPAN = 80 # Each ant lives for 80 iterations (requirement #5)

# Colors for visualization (RGB)
ANT_COLORS = {
    'queen': (255, 215, 0), # Gold
    'worker': (139, 69, 19), # Brown
    'reproducer': (0, 128, 0), # Green
    'soldier': (192, 0, 0) # Crimson red
}

# =====
# LANGTON RULES (requirement #2 - hidden cell state)
# =====
# Standard Langton ant rule:
# - On white cell (1): turn right, flip to black
# - On black cell (0): turn left, flip to white
DIRECTIONS = {
    'N': (0, -1), # North (up)
    'E': (1, 0), # East (right)
    'S': (0, 1), # South (down)
    'W': (-1, 0) # West (left)
}

DIRECTION_NAMES = ['N', 'E', 'S', 'W']
DIRECTION_TURNS = {
    'N': {'L': 'W', 'R': 'E'},
    'E': {'L': 'N', 'R': 'S'},
    'S': {'L': 'E', 'R': 'W'},
    'W': {'L': 'S', 'R': 'N'}
}

```



```

# =====
# REPRODUCTION RULES (requirement #4)
# =====
REPRODUCTION_PROBABILITY = 0.5 # 50% chance reproducer + queen reproduce
# Queen and reproducer must have opposite facing directions (180ř)
OPPOSITE_DIRECTIONS = {
    'N': 'S',
    'S': 'N',
    'E': 'W',
    'W': 'E'
}

# =====
# DEATH RULES
# =====

# Queen encounter rule (requirement #6)
QUEEN_ENCOUNTER_SAME_AGE_PROB = 0.5 # Both age < 60: 50% survival
QUEEN_ENCOUNTER_OLD_AGE_THRESHOLD = 60 # Age threshold for old queens
QUEEN_ENCOUNTER_OLD_AGE_PROB = 0.2 # Age >= 60: 20% survival for both

# Collision handling (requirement #7)
# If target cell occupied, try 3 other directions randomly
# If all 4 directions occupied, wait (no movement this iteration)

# =====
# SIMULATION PARAMETERS
# =====
DEFAULT_ITERATIONS = 300 # Default number of iterations to run
RANDOM_SEED = 42 # For reproducibility during development

# =====
# SCENARIO DETECTION THRESHOLDS
# =====
# For requirement #9: identify three scenarios

# Death by isolation: population drops below X% of initial
ISOLATION_THRESHOLD = 0.10 # 10% of initial population

# Overpopulation: grid occupancy > X% AND rapid death rate
OVERPOPULATION_GRID_THRESHOLD = 0.80 # 80% grid occupancy
OVERPOPULATION_DEATH_RATE = 0.15 # Death rate > 15% per iteration

# Stable state: population within X% for Y consecutive generations
STABLE_POPULATION_VARIANCE = 0.05 # ±5%
STABLE_GENERATIONS_REQUIRED = 50 # 50 consecutive generations

```

7.2 Archivo ant.py

Listing 2: ant.py

```
"""
```

```

Ant class hierarchy for Langton's multi-ant simulation.
Base class + 4 subclasses: Queen, Worker, Reproducer, Soldier
"""

```

```

from config import ANT_COLORS, DIRECTIONS, DIRECTION_NAMES, LIFESPAN

```

```

class Ant:
    """Base Ant class with shared properties and methods."""

    def __init__(self, x, y, ant_type, direction='N'):
        """
        Initialize an ant.

        Args:
            x, y: Grid position
            ant_type: One of 'queen', 'worker', 'reproducer', 'soldier'
            direction: One of 'N', 'E', 'S', 'W' (default: North)
        """
        self.x = x
        self.y = y
        self.ant_type = ant_type
        self.direction = direction
        self.age = 0
        self.lifespan = LIFESPAN
        self.color = ANT_COLORS.get(ant_type, (128, 128, 128))

    def age_one_iteration(self):
        """Increment age by 1 iteration."""
        self.age += 1

    def is_alive(self):
        """Check if ant is still alive (age < lifespan)."""
        return self.age < self.lifespan

    def move(self, new_x, new_y):
        """Move ant to new position."""
        self.x = new_x
        self.y = new_y

    def turn(self, direction):
        """Turn ant to face new direction."""
        self.direction = direction

    def get_position(self):
        """Return (x, y) tuple."""
        return (self.x, self.y)

    def get_direction_offset(self):
        """Return (dx, dy) offset for current facing direction."""
        return DIRECTIONS[self.direction]

```

```

def __repr__(self):
    return f"Ant({self.ant_type}, age={self.age}, pos=({self.x},{self.y}),
        dir={self.direction})"

class Queen(Ant):
    """Queen ant: can reproduce with reproducer ants."""

    def __init__(self, x, y, direction='N'):
        super().__init__(x, y, 'queen', direction)

class Worker(Ant):
    """Worker ant: standard moving ant, no special behavior."""

    def __init__(self, x, y, direction='N'):
        super().__init__(x, y, 'worker', direction)

class Reproducer(Ant):
    """Reproducer ant: can reproduce with queen ants (opposite facing
        direction)."""

    def __init__(self, x, y, direction='N'):
        super().__init__(x, y, 'reproducer', direction)

class Soldier(Ant):
    """Soldier ant: standard moving ant, no special behavior."""

    def __init__(self, x, y, direction='N'):
        super().__init__(x, y, 'soldier', direction)

# Factory function to create ant by type
def create_ant(x, y, ant_type, direction='N'):
    """
    Factory function to create an ant of the specified type.

    Args:
        x, y: Grid position
        ant_type: One of 'queen', 'worker', 'reproducer', 'soldier'
        direction: One of 'N', 'E', 'S', 'W'

    Returns:
        Ant instance of the specified type
    """
    ant_classes = {
        'queen': Queen,
        'worker': Worker,

```

```

        'reproducer': Reproducer,
        'soldier': Soldier
    }

    if ant_type not in ant_classes:
        raise ValueError(f"Unknown ant type: {ant_type}")

    return ant_classes[ant_type](x, y, direction)

```

7.3 Archivo grid.py

Listing 3: grid.py

```

"""
Grid class managing the simulation world state.
Handles:
- Cell occupancy tracking
- Hidden Langton cell state (black=0, white=1)
- Langton movement rules
- Ant placement, movement, and removal
"""

import numpy as np
import random
from config import GRID_SIZE, DIRECTIONS, DIRECTION_TURNS, DIRECTION_NAMES

class Grid:
    """Represents the simulation world."""

    def __init__(self, grid_size=GRID_SIZE, seed=None, toroidal=True):
        """
        Initialize the grid.

        Args:
            grid_size: Width/height of the grid.
            seed: Random seed for reproducibility
        """
        self.grid_size = grid_size
        self.toroidal = toroidal

        if seed is not None:
            random.seed(seed)
            np.random.seed(seed)

        # Cell occupancy: True if an ant is present, False otherwise
        self.occupancy = np.zeros((self.grid_size, self.grid_size), dtype=bool)

        # Langton cell state: 0=black (turn left), 1=white (turn right)
        # Initialized randomly (50% black, 50% white) per requirement #2
        self.cell_state = np.random.choice([0, 1], size=(self.grid_size,
            self.grid_size))

```

```

    # Store ants: dict mapping (x, y) -> Ant object
    # Max 1 ant per cell per requirement #7
    self.ants = {}

def is_valid_position(self, x, y):
    """Check if position is within grid bounds."""
    return 0 <= x < self.grid_size and 0 <= y < self.grid_size

def get_ant(self, x, y):
    """Get ant at position (x, y), or None if empty."""
    return self.ants.get((x, y), None)

def place_ant(self, ant, x, y):
    """
    Place an ant at position (x, y).

    Args:
        ant: Ant object
        x, y: Grid position

    Returns:
        True if placement successful, False if cell occupied
    """
    if not self.is_valid_position(x, y):
        return False

    if self.occupancy[y, x]:
        return False # Cell already occupied

    ant.move(x, y)
    self.ants[(x, y)] = ant
    self.occupancy[y, x] = True
    return True

def remove_ant(self, x, y):
    """Remove ant at position (x, y)."""
    if (x, y) in self.ants:
        del self.ants[(x, y)]
    self.occupancy[y, x] = False

def move_ant(self, ant, new_x, new_y):
    """
    Move ant to new position.

    Args:
        ant: Ant object
        new_x, new_y: Target position

    Returns:
        True if move successful, False if target occupied
    """

```

```

"""
if not self.is_valid_position(new_x, new_y):
    return False

if self.occupancy[new_y, new_x]:
    return False # Target occupied

# Remove from old position
old_x, old_y = ant.x, ant.y
if (old_x, old_y) in self.ants:
    del self.ants[(old_x, old_y)]
self.occupancy[old_y, old_x] = False

# Place at new position
ant.move(new_x, new_y)
self.ants[(new_x, new_y)] = ant
self.occupancy[new_y, new_x] = True
return True

def flip_cell_state(self, x, y):
    """Flip cell state from black (0) to white (1) or vice versa."""
    if self.is_valid_position(x, y):
        self.cell_state[y, x] = 1 - self.cell_state[y, x]

def get_cell_state(self, x, y):
    """Get cell state at (x, y): 0=black, 1=white."""
    if self.is_valid_position(x, y):
        return self.cell_state[y, x]
    return 1 # Default to white if out of bounds

def apply_langton_rule(self, ant):
    """
    Apply Langton's rule to an ant:
    1. Check current cell color
    2. Turn left (black) or right (white)
    3. Flip cell color
    4. Move forward in new direction

    Returns:
        (new_x, new_y) if move successful, None if blocked
    """
    x, y = ant.x, ant.y
    cell_color = self.get_cell_state(x, y)

    # Determine turn direction based on cell color
    turn_direction = 'L' if cell_color == 0 else 'R'

    # Turn ant
    new_direction = DIRECTION_TURNS[ant.direction][turn_direction]
    ant.turn(new_direction)

```

```

    # Flip cell color
    self.flip_cell_state(x, y)

    # Calculate forward position
    dx, dy = DIRECTIONS[ant.direction]
    new_x = x + dx
    new_y = y + dy

    if self.toroidal:
        new_x = new_x % self.grid_size
        new_y = new_y % self.grid_size
    else:
        if not self.is_valid_position(new_x, new_y):
            # Still apply turn and flip, but movement is out of bounds
            return None

    return (new_x, new_y)

def attempt_movement_with_collision(self, ant):
    """
    Try to move ant following Langton rule.
    If target cell is occupied, try 3 other random directions.
    If all 4 directions occupied, ant waits (no movement).

    Args:
        ant: Ant object

    Returns:
        True if ant moved, False if stuck (all directions occupied)
    """
    x, y = ant.x, ant.y
    current_direction = ant.direction

    # Apply Langton rule to get target position
    result = self.apply_langton_rule(ant)

    # Try to move to target (result may be None if out-of-bounds when not toroidal)
    if result is not None:
        target_x, target_y = result
        if not self.occupancy[target_y, target_x]:
            self.move_ant(ant, target_x, target_y)
            return True

    # Target occupied: try 3 other random directions
    # Get remaining 3 directions
    remaining_directions = [d for d in DIRECTION_NAMES if d !=
                             ant.direction]
    random.shuffle(remaining_directions)

    for direction in remaining_directions:

```

```

        ant.turn(direction)
        dx, dy = DIRECTIONS[direction]
        test_x = x + dx
        test_y = y + dy

        if self.toroidal:
            test_x = test_x % self.grid_size
            test_y = test_y % self.grid_size
        else:
            if not self.is_valid_position(test_x, test_y):
                continue

        if not self.occupancy[test_y, test_x]:
            self.move_ant(ant, test_x, test_y)
            return True

        # All directions occupied: revert to original direction and wait
        ant.turn(current_direction)
        return False

def get_all_ants(self):
    """Return list of all ants in the grid."""
    return list(self.ants.values())

def get_ant_count(self):
    """Return total number of ants."""
    return len(self.ants)

def get_occupancy_ratio(self):
    """Return fraction of occupied cells."""
    return np.sum(self.occupancy) / (self.grid_size * self.grid_size)

def get_ant_by_type(self, ant_type):
    """Return list of all ants of a specific type."""
    return [ant for ant in self.ants.values() if ant.ant_type == ant_type]

def get_state_grid(self):
    """
    Return a grid for visualization: each cell value is the ant color (if
    present).
    Used by visualization to render the world.

    Returns:
        2D numpy array of RGB tuples (or zeros for empty cells)
    """
    state_grid = np.zeros((self.grid_size, self.grid_size, 3),
                          dtype=np.uint8)

    for (x, y), ant in self.ants.items():
        state_grid[y, x] = ant.color

```



```
return state_grid
```

7.4 Archivo simulation.py

Listing 4: simulation.py

```
"""
Simulation class orchestrating the main simulation loop.
Handles:
- Grid initialization with ant distribution
- Iteration loop (movement reproduction aging death)
- Statistics tracking
- Scenario detection (isolation, overpopulation, stable)
"""

import random
import numpy as np
from config import (
    GRID_SIZE, OCCUPANCY_RATIO, ANT_DENSITIES, LIFESPAN,
    REPRODUCTION_PROBABILITY, OPPOSITE_DIRECTIONS,
    QUEEN_ENCOUNTER_SAME_AGE_PROB,
    QUEEN_ENCOUNTER_OLD_AGE_THRESHOLD, QUEEN_ENCOUNTER_OLD_AGE_PROB,
    ISOLATION_THRESHOLD, OVERPOPULATION_GRID_THRESHOLD,
    OVERPOPULATION_DEATH_RATE,
    STABLE_POPULATION_VARIANCE, STABLE_GENERATIONS_REQUIRED
)
from grid import Grid
from ant import create_ant

class Simulation:
    """Main simulation orchestrator."""

    def __init__(self, grid_size=GRID_SIZE, occupancy_ratio=OCCUPANCY_RATIO,
        seed=None, toroidal=True):
        """
        Initialize the simulation.

        Args:
            grid_size: Size of grid (grid_size x grid_size)
            occupancy_ratio: Fraction of grid initially occupied
            seed: Random seed for reproducibility
        """
        self.grid = Grid(grid_size=grid_size, seed=seed, toroidal=toroidal)
        self.toroidal = toroidal
        self.grid_size = grid_size
        self.generation = 0
        self.initial_ant_count = 0

        # Statistics tracking
        self.stats = {
            'generation': [],
```

```

        'total_ants': [],
        'queen_count': [],
        'worker_count': [],
        'reproducer_count': [],
        'soldier_count': [],
        'births': [],
        'deaths': [],
        'avg_age': [],
        'occupancy_ratio': []
    }

    # Initialize grid with ants
    self._initialize_ants(occupancy_ratio)
    self.initial_ant_count = self.grid.get_ant_count()

def _initialize_ants(self, occupancy_ratio):
    """
    Initialize the grid with ants following the density distribution.

    Args:
        occupancy_ratio: Fraction of grid cells to initially occupy
                        (requirement #3)
    """
    # Calculate number of ants to place
    total_cells = self.grid_size * self.grid_size
    num_ants_to_place = int(total_cells * occupancy_ratio)

    # Shuffle all positions in the grid
    positions = [(x, y) for x in range(self.grid_size) for y in
                  range(self.grid_size)]
    random.shuffle(positions)
    positions = positions[:num_ants_to_place]

    # Assign ant types based on densities
    ant_type_counts = {
        'queen': int(num_ants_to_place * ANT_DENSITIES['queen']),
        'worker': int(num_ants_to_place * ANT_DENSITIES['worker']),
        'reproducer': int(num_ants_to_place * ANT_DENSITIES['reproducer']),
        'soldier': int(num_ants_to_place * ANT_DENSITIES['soldier'])
    }

    # Adjust for rounding errors
    total_assigned = sum(ant_type_counts.values())
    if total_assigned < num_ants_to_place:
        ant_type_counts['worker'] += (num_ants_to_place - total_assigned)

    # Create ant type list (shuffled)
    ant_types = []
    for ant_type, count in ant_type_counts.items():
        ant_types.extend([ant_type] * count)
    random.shuffle(ant_types)

```

```

    # Place ants
    for i, (x, y) in enumerate(positions):
        if i < len(ant_types):
            ant_type = ant_types[i]
            direction = random.choice(['N', 'E', 'S', 'W'])
            ant = create_ant(x, y, ant_type, direction)
            self.grid.place_ant(ant, x, y)

def iterate(self):
    """Execute one simulation iteration."""
    self.generation += 1

    births = 0
    deaths = 0

    # Phase 1: Movement (apply Langton rules)
    ants_to_process = self.grid.get_all_ants().copy()
    for ant in ants_to_process:
        if (ant.x, ant.y) in self.grid.ants: # Verify ant still exists
            self.grid.attempt_movement_with_collision(ant)

    # Phase 2: Reproduction (requirement #4) - optimized with spatial search
    # Only check reproducers that could be near queens (within Manhattan distance 1)
    reproducers = self.grid.get_ant_by_type('reproducer')

    for reproducer in reproducers:
        if (reproducer.x, reproducer.y) not in self.grid.ants:
            continue # Reproducer dead

        # Check only adjacent positions for queens
        adjacent_positions = [
            (reproducer.x - 1, reproducer.y - 1), (reproducer.x,
            reproducer.y - 1), (reproducer.x + 1, reproducer.y - 1),
            (reproducer.x - 1, reproducer.y), (reproducer.x, reproducer.y),
            (reproducer.x + 1, reproducer.y),
            (reproducer.x - 1, reproducer.y + 1), (reproducer.x,
            reproducer.y + 1), (reproducer.x + 1, reproducer.y + 1)
        ]

        for adj_x, adj_y in adjacent_positions:
            # Wrap coordinates
            adj_x = adj_x % self.grid_size
            adj_y = adj_y % self.grid_size

            queen = self.grid.get_ant(adj_x, adj_y)
            if queen and queen.ant_type == 'queen':
                # Check if they have opposite facing directions
                if reproducer.direction ==

```

```

OPPOSITE_DIRECTIONS.get(queen.direction):
# 50% probability to reproduce
if random.random() < REPRODUCTION_PROBABILITY:
    # Spawn new ant in an adjacent free cell.
    # Prefer positions that are adjacent to both queen
    # and reproducer when possible.
    new_ant_type = random.choice(['queen', 'worker',
                                  'reproducer', 'soldier'])
    new_direction = random.choice(['N', 'E', 'S', 'W'])

def neighbor_positions(x, y):
    positions = []
    for dx in [-1, 0, 1]:
        for dy in [-1, 0, 1]:
            if dx == 0 and dy == 0:
                continue
            nx = (x + dx) % self.grid_size
            ny = (y + dy) % self.grid_size
            positions.append((nx, ny))
    return positions

reproducer_neighbors =
    set(neighbor_positions(reproducer.x,
                           reproducer.y))
queen_neighbors = set(neighbor_positions(queen.x,
                                          queen.y))
shared_neighbors = list(reproducer_neighbors &
                        queen_neighbors)
candidate_positions = []

if shared_neighbors:
    random.shuffle(shared_neighbors)
    candidate_positions.extend(shared_neighbors)

remaining = list(reproducer_neighbors -
                 set(candidate_positions))
random.shuffle(remaining)
candidate_positions.extend(remaining)

# As a fallback, allow positions adjacent to the
# queen as well
fallback = list(queen_neighbors -
                set(candidate_positions))
random.shuffle(fallback)
candidate_positions.extend(fallback)

placed = False
for new_x, new_y in candidate_positions:
    if not self.grid.occupancy[new_y, new_x]:
        new_ant = create_ant(new_x, new_y,
                              new_ant_type, new_direction)

```

```

        self.grid.place_ant(new_ant, new_x, new_y)
        births += 1
        placed = True
        break

# Phase 3: Aging
for ant in self.grid.get_all_ants():
    ant.age_one_iteration()

# Phase 4: Death by age (requirement #5 - 80 iteration lifespan)
ants_to_remove = []
for ant in self.grid.get_all_ants():
    if not ant.is_alive():
        ants_to_remove.append(ant)

for ant in ants_to_remove:
    self.grid.remove_ant(ant.x, ant.y)
    deaths += 1

# Phase 5: Queen encounter rules (requirement #6) - optimized spatial
check
queens = self.grid.get_ant_by_type('queen')
checked_pairs = set() # Track checked queen pairs to avoid duplicates

for i, queen1 in enumerate(queens):
    if (queen1.x, queen1.y) not in self.grid.ants:
        continue # Queen already dead

    # Only check adjacent queen locations (not all pairs)
    adjacent_positions = [
        (queen1.x - 1, queen1.y - 1), (queen1.x, queen1.y - 1),
        (queen1.x + 1, queen1.y - 1),
        (queen1.x - 1, queen1.y), (queen1.x + 1, queen1.y),
        (queen1.x - 1, queen1.y + 1), (queen1.x, queen1.y + 1),
        (queen1.x + 1, queen1.y + 1)
    ]

    for adj_x, adj_y in adjacent_positions:
        # Wrap coordinates
        adj_x = adj_x % self.grid_size
        adj_y = adj_y % self.grid_size

        queen2 = self.grid.get_ant(adj_x, adj_y)
        if queen2 and queen2.ant_type == 'queen' and queen1 is not
            queen2:
            # Create a consistent pair identifier
            pair_id = (min(id(queen1), id(queen2)), max(id(queen1),
                id(queen2)))
            if pair_id in checked_pairs:
                continue
            checked_pairs.add(pair_id)

```

```

        # They meet: apply rule
        if queen1.age < QUEEN_ENCOUNTER_OLD_AGE_THRESHOLD and \
            queen2.age < QUEEN_ENCOUNTER_OLD_AGE_THRESHOLD:
            # Both young: 50% probability one dies
            if random.random() < QUEEN_ENCOUNTER_SAME_AGE_PROB:
                victim = random.choice([queen1, queen2])
                if (victim.x, victim.y) in self.grid.ants:
                    self.grid.remove_ant(victim.x, victim.y)
                    deaths += 1
            else:
                # At least one old: 20% probability both die
                if random.random() < QUEEN_ENCOUNTER_OLD_AGE_PROB:
                    for queen in [queen1, queen2]:
                        if (queen.x, queen.y) in self.grid.ants:
                            self.grid.remove_ant(queen.x, queen.y)
                            deaths += 1

    # Record statistics
    self._record_stats(births, deaths)

def _record_stats(self, births, deaths):
    """Record statistics for this iteration."""
    all_ants = self.grid.get_all_ants()
    type_counts = {
        'queen': len([a for a in all_ants if a.ant_type == 'queen']),
        'worker': len([a for a in all_ants if a.ant_type == 'worker']),
        'reproducer': len([a for a in all_ants if a.ant_type ==
            'reproducer']),
        'soldier': len([a for a in all_ants if a.ant_type == 'soldier'])
    }

    avg_age = np.mean([a.age for a in all_ants]) if all_ants else 0

    self.stats['generation'].append(self.generation)
    self.stats['total_ants'].append(len(all_ants))
    self.stats['queen_count'].append(type_counts['queen'])
    self.stats['worker_count'].append(type_counts['worker'])
    self.stats['reproducer_count'].append(type_counts['reproducer'])
    self.stats['soldier_count'].append(type_counts['soldier'])
    self.stats['births'].append(births)
    self.stats['deaths'].append(deaths)
    self.stats['avg_age'].append(avg_age)
    self.stats['occupancy_ratio'].append(self.grid.get_occupancy_ratio())

def detect_scenario(self):
    """
    Detect which scenario the system is in.

    Returns:
        'isolation', 'overpopulation', 'stable', or None
    """

```

```

"""
if len(self.stats['total_ants']) < 2:
    return None

current_pop = self.stats['total_ants'][-1]

# Isolation: population drops below 10% of initial
if current_pop < self.initial_ant_count * ISOLATION_THRESHOLD:
    return 'isolation'

# Overpopulation: grid occupancy > 80% AND rapid death rate
if len(self.stats['occupancy_ratio']) > 0:
    if self.stats['occupancy_ratio'][-1] >
        OVERPOPULATION_GRID_THRESHOLD:
        if len(self.stats['deaths']) > 0 and self.stats['deaths'][-1] >
            0:
            death_rate = self.stats['deaths'][-1] / (current_pop +
                self.stats['deaths'][-1]) \
                if (current_pop + self.stats['deaths'][-1]) > 0 else 0
            if death_rate > OVERPOPULATION_DEATH_RATE:
                return 'overpopulation'

# Stable state: population within ±5% for 50+ consecutive generations
if len(self.stats['total_ants']) >= STABLE_GENERATIONS_REQUIRED:
    recent_pops =
        self.stats['total_ants'][-STABLE_GENERATIONS_REQUIRED:]
    avg_pop = np.mean(recent_pops)
    variance = np.std(recent_pops) / avg_pop if avg_pop > 0 else 0

    if variance <= STABLE_POPULATION_VARIANCE:
        return 'stable'

return None

def run(self, num_iterations):
    """
    Run the simulation for a specified number of iterations.

    Args:
        num_iterations: Number of iterations to run
    """
    for _ in range(num_iterations):
        self.iterate()

def get_stats(self):
    """Return statistics dictionary."""
    return self.stats

def get_grid(self):
    """Return the grid object."""
    return self.grid

```

7.5 Archivo main.py

Listing 5: main.py

```
"""
Main entry point for Langton's multi-ant simulation.
Provides CLI interface for running simulations with various options.
"""

import argparse
import os
import sys
from datetime import datetime
from simulation import Simulation
from visualization import SimulationVisualizer
from config import DEFAULT_ITERATIONS, RANDOM_SEED

def main():
    """Main CLI entry point."""
    parser = argparse.ArgumentParser(
        description='Langton\'s Multi-Ant Simulation',
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog="""
Examples:
python main.py --iterations 500 --visualize
python main.py --iterations 1000 --report --seed 42
python main.py --batch 3 --iterations 200 # Run 3 simulations
        """
    )

    parser.add_argument(
        '--iterations', '-i', type=int, default=DEFAULT_ITERATIONS,
        help=f'Number of iterations to run (default: {DEFAULT_ITERATIONS})'
    )

    parser.add_argument(
        '--seed', '-s', type=int, default=RANDOM_SEED,
        help=f'Random seed for reproducibility (default: {RANDOM_SEED})'
    )

    parser.add_argument(
        '--grid-size', '-g', type=int, default=100,
        help='Grid size (N x N, default: 100)'
    )

    parser.add_argument(
        '--occupancy', '-o', type=float, default=0.5,
        help='Initial occupancy ratio (0-1, default: 0.5 for 50%)'
    )

    parser.add_argument(
```



```

        '--visualize', '-v', action='store_true',
        help='Run interactive real-time visualization'
    )

    parser.add_argument(
        '--toroidal', dest='toroidal', action='store_true',
        help='Enable toroidal wrap-around (default)'
    )
    parser.add_argument(
        '--no-toroidal', dest='toroidal', action='store_false',
        help='Disable toroidal wrap; edges are boundaries'
    )
    parser.set_defaults(toroidal=True)

    parser.add_argument(
        '--report', '-r', action='store_true',
        help='Generate final report with plots'
    )

    parser.add_argument(
        '--csv', '-c', action='store_true',
        help='Export statistics to CSV'
    )

    parser.add_argument(
        '--output-dir', type=str, default='output',
        help='Output directory for reports (default: output)'
    )

    parser.add_argument(
        '--batch', '-b', type=int, default=1,
        help='Run multiple simulations (for parameter exploration)'
    )

    parser.add_argument(
        '--log-file', type=str, default=None,
        help='Optional file to save console output summary'
    )

    parser.add_argument(
        '--quiet', '-q', action='store_true',
        help='Suppress status messages'
    )

    args = parser.parse_args()

    # Open log file if requested
    log_file_handle = None
    if args.log_file:
        os.makedirs(os.path.dirname(args.log_file) or '.', exist_ok=True)
        log_file_handle = open(args.log_file, 'w', encoding='utf-8')

```

```

def log(message=''):
    if not args.quiet:
        print(message)
    if log_file_handle is not None:
        log_file_handle.write(f"{message}\n")

# Validate arguments
if not (0 < args.occupancy <= 1):
    log("Error: Occupancy must be between 0 and 1")
    if log_file_handle is not None:
        log_file_handle.close()
    sys.exit(1)

if args.iterations < 1:
    print("Error: Iterations must be at least 1")
    sys.exit(1)

def build_run_name(seed, grid_size, iterations, occupancy, toroidal):
    occupancy_str = f"{occupancy:.2f}".rstrip('0').rstrip('.')
    toroidal_tag = 'toroidal' if toroidal else 'notor'
    timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
    return
        f"S{seed}_G{grid_size}_I{iterations}_O{occupancy_str}_{toroidal_tag}_{timestamp}"

# Run simulation(s)
for batch_num in range(args.batch):
    if args.batch > 1:
        log(f"\n{'='*60}")
        log(f"Run {batch_num + 1}/{args.batch}")
        log(f"{'='*60}")

    # Create simulation
    seed = args.seed + batch_num if args.batch > 1 else args.seed
    sim = Simulation(
        grid_size=args.grid_size,
        occupancy_ratio=args.occupancy,
        seed=seed,
        toroidal=args.toroidal
    )
    run_name = build_run_name(seed, args.grid_size, args.iterations,
        args.occupancy, args.toroidal)

    log(f"Initialized grid {args.grid_size}x{args.grid_size}")
    log(f"Initial ants: {sim.initial_ant_count}")
    log(f"Occupancy: {sim.grid.get_occupancy_ratio():.2%}")

    # Create visualizer
    visualizer = SimulationVisualizer(sim, output_dir=args.output_dir)

    # Run visualization or headless simulation

```

```

if args.visualize:
    log(f"Starting interactive visualization for {args.iterations}
        iterations...")
    visualizer.run_interactive_animation(num_generations=args.iterations)
else:
    log(f"Running simulation for {args.iterations} iterations...")
    sim.run(args.iterations)

    final_scenario = sim.detect_scenario()
    log(f"Final ant count: {sim.grid.get_ant_count()}")
    log(f"Final occupancy: {sim.grid.get_occupancy_ratio():.2%}")
    if len(sim.stats['generation']) >= 80:
        log(f"Iteración 80: {sim.stats['total_ants'][79]} hormigas")
    else:
        log(f"Iteración 80: no disponible (solo
            {len(sim.stats['generation'])} iteraciones)")
    if final_scenario:
        log(f"Detected scenario: {final_scenario}")

    # Generate reports
    if args.report:
        log("Generating report...")
        visualizer.generate_scenario_reports(run_name=run_name)

    if args.csv:
        log("Exporting statistics...")
        visualizer.export_statistics_csv(filename=f'{run_name}.csv')

log("\nDone!")
if log_file_handle is not None:
    log_file_handle.close()

if __name__ == '__main__':
    main()

```

7.6 Archivo visualization.py

Listing 6: visualization.py

```

"""
Visualization module for Langton's ant simulation.
Handles:
- Real-time matplotlib animation
- Live statistics display
- Report generation (plots for isolation, overpopulation, stable scenarios)
"""

import matplotlib.pyplot as plt
import matplotlib.animation as animation
from matplotlib.figure import Figure
import numpy as np

```

```

import os
from datetime import datetime

class SimulationVisualizer:
    """Handles real-time visualization and report generation."""

    def __init__(self, simulation, output_dir='output'):
        """
        Initialize the visualizer.

        Args:
            simulation: Simulation object
            output_dir: Directory to save reports
        """
        self.simulation = simulation
        self.output_dir = output_dir
        self.is_running = False
        self.speed = 1 # Speed multiplier for animation

        # Create output directory
        os.makedirs(output_dir, exist_ok=True)

        # Capture initial grid state before the simulation runs
        self.initial_grid_state =
            np.copy(self.simulation.grid.get_state_grid())

        # Animation figure
        self.fig = None
        self.axes = None
        self.anim = None

    def setup_animation_figure(self):
        """Set up matplotlib figure for real-time animation."""
        self.fig = plt.figure(figsize=(16, 10))

        # Main grid subplot
        ax_grid = plt.subplot(2, 3, (1, 4))

        # Statistics subplots
        ax_pop = plt.subplot(2, 3, 2)
        ax_age = plt.subplot(2, 3, 3)
        ax_types = plt.subplot(2, 3, 5)
        ax_occupancy = plt.subplot(2, 3, 6)

        self.axes = {
            'grid': ax_grid,
            'population': ax_pop,
            'age': ax_age,
            'types': ax_types,
            'occupancy': ax_occupancy

```

```

}

self.fig.suptitle('Langton\'s Multi-Ant Simulation', fontsize=16)
return self.fig

def update_animation_frame(self, frame):
    """Update visualization for one animation frame."""
    # Run simulation steps
    for _ in range(self.speed):
        self.simulation.iterate()
        scenario = self.simulation.detect_scenario()
        if scenario:
            print(f"[Generation {self.simulation.generation}] Scenario
                  detected: {scenario}")

    # Clear all subplots
    for ax in self.axes.values():
        ax.clear()

    # Update grid visualization
    self._plot_grid()

    # Update statistics plots
    self._plot_statistics()

    # Update title with current generation and info
    ant_count = self.simulation.grid.get_ant_count()
    scenario = self.simulation.detect_scenario()
    scenario_text = f" [SCENARIO: {scenario}]" if scenario else ""
    self.fig.suptitle(
        f'Langton\'s Multi-Ant Simulation - Gen
          {self.simulation.generation}, '
        f'Ants: {ant_count}{scenario_text}',
        fontsize=14
    )

    plt.tight_layout(rect=[0, 0, 1, 0.96])

def _plot_grid(self):
    """Plot the grid with ants."""
    ax = self.axes['grid']
    grid_state = self.simulation.grid.get_state_grid()
    ax.imshow(grid_state, origin='upper')
    ax.set_title('Grid (Ant Positions)')
    ax.set_xlabel('X')
    ax.set_ylabel('Y')

def _plot_statistics(self):
    """Plot all statistics subplots."""
    stats = self.simulation.stats
    generations = stats['generation']

```

```

if not generations:
    return

# Population over time
ax_pop = self.axes['population']
ax_pop.plot(generations, stats['total_ants'], 'k-', label='Total',
            linewidth=2)
ax_pop.plot(generations, stats['queen_count'], 'gold', label='Queen')
ax_pop.plot(generations, stats['worker_count'], 'brown',
            label='Worker')
ax_pop.plot(generations, stats['reproducer_count'], 'green',
            label='Reproducer')
ax_pop.plot(generations, stats['soldier_count'], 'red',
            label='Soldier')
ax_pop.set_title('Population Over Time')
ax_pop.set_xlabel('Generation')
ax_pop.set_ylabel('Count')
ax_pop.legend(fontsize=8)
ax_pop.grid(True, alpha=0.3)

# Age distribution
ax_age = self.axes['age']
all_ants = self.simulation.grid.get_all_ants()
if all_ants:
    ages = [ant.age for ant in all_ants]
    ax_age.hist(ages, bins=20, edgecolor='black', alpha=0.7)
    ax_age.set_title('Age Distribution (Current)')
    ax_age.set_xlabel('Age (iterations)')
    ax_age.set_ylabel('Frequency')
    ax_age.grid(True, alpha=0.3, axis='y')

# Type distribution (pie chart)
ax_types = self.axes['types']
type_counts = [
    stats['queen_count'][-1],
    stats['worker_count'][-1],
    stats['reproducer_count'][-1],
    stats['soldier_count'][-1]
]
labels = ['Queen', 'Worker', 'Reproducer', 'Soldier']
colors = ['gold', 'brown', 'green', 'red']
if sum(type_counts) > 0:
    ax_types.pie(type_counts, labels=labels, colors=colors,
                autopct='%1.1f%%')
    ax_types.set_title('Type Distribution (Current)')

# Occupancy ratio
ax_occupancy = self.axes['occupancy']
ax_occupancy.plot(generations, stats['occupancy_ratio'], 'b-',
                linewidth=2)

```

```

ax_occupancy.axhline(y=0.5, color='r', linestyle='--', label='Initial
    Target')
ax_occupancy.set_title('Grid Occupancy Ratio')
ax_occupancy.set_xlabel('Generation')
ax_occupancy.set_ylabel('Occupancy')
ax_occupancy.set_ylim([0, 1])
ax_occupancy.legend()
ax_occupancy.grid(True, alpha=0.3)

def run_interactive_animation(self, num_generations=500):
    """
    Run interactive real-time animation (requires display).

    Args:
        num_generations: Number of generations to animate
    """
    self.setup_animation_figure()

    # Create animation
    self.anim = animation.FuncAnimation(
        self.fig, self.update_animation_frame,
        frames=num_generations, repeat=False, interval=100
    )

    plt.show()

def save_grid_snapshot(self, grid_state, filename):
    """Save a grid snapshot image."""
    output_path = os.path.join(self.output_dir, filename)
    plt.imsave(output_path, grid_state)
    return output_path

def export_summary_text(self, run_name='scenario_report'):
    """Export a simple textual summary of the simulation results."""
    stats = self.simulation.stats
    summary_path = os.path.join(self.output_dir, f'{run_name}_summary.txt')
    final_generation = stats['generation'][-1] if stats['generation'] else
    0
    final_total = stats['total_ants'][-1] if stats['total_ants'] else 0
    final_queens = stats['queen_count'][-1] if stats['queen_count'] else 0
    final_workers = stats['worker_count'][-1] if stats['worker_count']
    else 0
    final_reproducers = stats['reproducer_count'][-1] if
    stats['reproducer_count'] else 0
    final_soldiers = stats['soldier_count'][-1] if stats['soldier_count']
    else 0
    final_occupancy = stats['occupancy_ratio'][-1] if
    stats['occupancy_ratio'] else 0
    final_scenario = self.simulation.detect_scenario() or 'None'

    with open(summary_path, 'w', encoding='utf-8') as f:

```

```

        f.write('Langton Multi-Ant Simulation Summary\n')
        f.write('=====\n')
        f.write(f'Run name: {run_name}\n')
        f.write(f'Generations: {final_generation}\n')
        f.write(f'Final total ants: {final_total}\n')
        f.write(f'Final queens: {final_queens}\n')
        f.write(f'Final workers: {final_workers}\n')
        f.write(f'Final reproducers: {final_reproducers}\n')
        f.write(f'Final soldiers: {final_soldiers}\n')
        f.write(f'Final occupancy ratio: {final_occupancy:.2%}\n')
        if len(stats['generation']) >= 80:
            f.write(f'Ants at generation 80: {stats['total_ants'][79]}\n')
        else:
            f.write(f'Ants at generation 80: n/a (only {final_generation}
                    generations)\n')
        f.write(f'Detected scenario: {final_scenario}\n')
    print(f'Summary exported: {summary_path}')
    return summary_path

def generate_scenario_reports(self, run_name='scenario_report'):
    """
    Generate final report with 3 scenario plots.

    Args:
        run_name: Name for the report files
    """
    stats = self.simulation.stats

    if not stats['generation']:
        print("No simulation data to report.")
        return

    # Create figure with 4 report subplots including initial/final
    # snapshots
    fig, axes = plt.subplots(2, 2, figsize=(18, 12))
    generations = stats['generation']

    # Plot 1: Initial grid state
    ax = axes[0, 0]
    ax.imshow(self.initial_grid_state, origin='upper')
    ax.set_title('Initial Grid State')
    ax.set_xlabel('X')
    ax.set_ylabel('Y')

    # Plot 2: Final grid state visualization
    ax = axes[0, 1]
    final_grid_state = self.simulation.grid.get_state_grid()
    ax.imshow(final_grid_state, origin='upper')
    ax.set_title('Final Grid State')
    ax.set_xlabel('X')
    ax.set_ylabel('Y')

```



```

# Plot 3: Population dynamics over time
ax = axes[1, 0]
ax.plot(generations, stats['total_ants'], 'k-', label='Total',
        linewidth=2)
ax.plot(generations, stats['queen_count'], 'gold', label='Queen')
ax.plot(generations, stats['worker_count'], 'brown', label='Worker')
ax.plot(generations, stats['reproducer_count'], 'green',
        label='Reproducer')
ax.plot(generations, stats['soldier_count'], 'red', label='Soldier')
ax.set_title('Population Dynamics')
ax.set_xlabel('Generation')
ax.set_ylabel('Count')
ax.legend(fontsize=8)
ax.grid(True, alpha=0.3)

# Plot 4: Occupancy ratio over time
ax = axes[1, 1]
ax.plot(generations, stats['occupancy_ratio'], 'b-', linewidth=2)
ax.fill_between(generations, stats['occupancy_ratio'], alpha=0.3)
ax.set_title('Grid Occupancy Over Time')
ax.set_xlabel('Generation')
ax.set_ylabel('Occupancy Ratio')
ax.set_ylim([0, 1])
ax.grid(True, alpha=0.3)

iteration_80_text = (
    f"Ants at generation 80: {stats['total_ants'][79]}\n"
    if len(generations) >= 80 else
    f"Ants at generation 80: n/a (only {generations[-1]} generations)\n"
)
summary_text = (
    f"Generations: {generations[-1]}\n"
    f"Final ants: {stats['total_ants'][-1]}\n"
    f"Queens: {stats['queen_count'][-1]}, Workers: {stats['worker_count'][-1]}\n"
    f"Reproducers: {stats['reproducer_count'][-1]}, Soldiers: {stats['soldier_count'][-1]}\n"
    f"Final occupancy: {stats['occupancy_ratio'][-1]:.2%}\n"
    f"{iteration_80_text}"
    f"Detected scenario: {self.simulation.detect_scenario() or 'None'}"
)
fig.text(0.5, 0.02, summary_text, ha='center', fontsize=12)

plt.suptitle('Langton\'s Multi-Ant Simulation - Initial and Final Report', fontsize=16)
plt.tight_layout(rect=[0, 0.05, 1, 0.96])

if run_name:
    report_filename = f'{run_name}.png'
else:

```

```

        timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
        report_filename = f'scenario_report_{timestamp}.png'
        report_path = os.path.join(self.output_dir, report_filename)
        plt.savefig(report_path, dpi=150, bbox_inches='tight')
        plt.close(fig)
        print(f"Report saved: {report_path}")

        # Save separate initial and final snapshots for easier comparison
        initial_snapshot = self.save_grid_snapshot(self.initial_grid_state,
            f'{run_name}_initial.png')
        final_snapshot = self.save_grid_snapshot(final_grid_state,
            f'{run_name}_final.png')
        print(f"Initial grid snapshot saved: {initial_snapshot}")
        print(f"Final grid snapshot saved: {final_snapshot}")

        # Generate a summary text file with key metrics
        self.export_summary_text(run_name)

    return report_path

def export_statistics_csv(self, filename='simulation_stats.csv'):
    """
    Export statistics to CSV file.

    Args:
        filename: Output CSV filename
    """
    import csv

    stats = self.simulation.stats
    output_path = os.path.join(self.output_dir, filename)

    with open(output_path, 'w', newline='') as f:
        writer = csv.DictWriter(f, fieldnames=stats.keys())
        writer.writeheader()

        # Transpose stats dictionary to write rows
        num_rows = len(stats['generation'])
        for i in range(num_rows):
            row = {key: stats[key][i] for key in stats.keys()}
            writer.writerow(row)

    print(f"Statistics exported: {output_path}")
    return output_path

```